Web Cache Poisoning & Web Cache Deception

These two vulnerability classes both abuse web caches, but they're fundamentally different attacks that get conflated constantly — worth locking in the distinction immediately: **cache poisoning** tricks the cache into storing a *malicious response* that other users then receive; **cache deception** tricks the cache into storing a *legitimate but private response* that an attacker then retrieves. One weaponizes the cache as a delivery mechanism for an attack; the other weaponizes it as a leak mechanism for private data. Both connect directly back to your XXE/SSRF host-header discussion and your Open Redirect/XSS delivery concepts from earlier guides.

---

1. How Web Caches Work — The Foundation

A web cache sits between the origin server and users, storing copies of responses so repeated requests for the same resource don't have to hit the backend every time.

```
1. Client requests a resource
2. Cache checks if it already has a stored copy
3. CACHE HIT  → cache serves the stored response directly, origin never touched
4. CACHE MISS → request is forwarded to the origin server, the response
                is returned to the client AND stored in the cache for
                next time
```

The cache key — the entire concept both vulnerabilities exploit: the cache doesn't store responses per-URL exactly — it generates a **key** from specific components of the request (typically the URL path, and sometimes select query parameters) to decide which stored response to serve for which incoming request. Critically, **most components of a request are NOT included in the cache key** — headers, cookies, and many query parameters are typically **unkeyed**, meaning their value can vary between requests without the cache treating them as different.

```
Keyed inputs     → determine which cached entry gets served (URL path,
                   sometimes specific query params)
Unkeyed inputs    → NOT part of the cache key — their value influences
```

```
                        the server's response, but the cache has no idea
                        they're different, and will happily serve the SAME
                        cached response regardless of what they contain
```

This single fact — unkeyed inputs influencing the response without influencing the cache key — is the entire root cause of web cache poisoning.

PART 1: Web Cache Poisoning

2. The Two-Phase Attack Model

```
Phase 1: Elicit a HARMFUL response from the backend server —
         get the server to include something dangerous in its response,
         triggered via an UNKEYED input the cache doesn't account for

Phase 2: Get that harmful response CACHED, so it's served to
         every subsequent visitor who requests that same cache key —
         turning a single malicious request into a persistent,
         self-propagating attack against everyone who visits afterward
```

Why this is so severe compared to a "normal" reflected vulnerability: a typical reflected XSS needs to trick each individual victim into clicking a crafted malicious link. A poisoned cache needs **zero victim interaction whatsoever** — the payload is served automatically to anyone who simply visits the site's normal, legitimate URL, since the cache has no way to distinguish the poisoned response from a genuine one.

3. Finding Unkeyed Inputs

Since unkeyed inputs are, by definition, invisible to the cache key, finding them requires systematic probing — manually testing every plausible header is tedious, which is exactly why **Param Miner** (a free Burp extension by the same researcher behind this whole technique) exists: it automates guessing header/cookie names and observes whether each one changes the server's response, flagging candidates worth testing further.

Classic unkeyed headers to test manually:

```
X-Forwarded-Host
X-Forwarded-Scheme
X-Forwarded-Proto
X-Original-URL
```

```
X-Rewrite-URL
X-Host
```

Detection methodology:

```
1. Send a request with a distinctive value in a candidate header:
   X-Forwarded-Host: canary-test-value.com
2. Check whether that value gets REFLECTED anywhere in the response
   (a link, a script src, a canonical URL, an error message)
3. If it IS reflected, you've found an unkeyed input that influences
   the response — a strong candidate for poisoning
4. Add a cache-buster query parameter to each test request
   (?cb=uniquevalue) so you're always hitting a fresh cache miss and
   seeing the server's genuine live behavior, not a previously cached result
```

4. Basic Attack — Unkeyed Header Reflection to XSS

This is the canonical, simplest form of the attack, and directly reuses techniques from your DOM XSS and Open Redirect guides:

```
GET / HTTP/1.1
Host: vulnerable-website.com
X-Forwarded-Host: "><script>alert(document.cookie)</script>
```

If the application reflects the `X-Forwarded-Host` value somewhere in its response (for instance, building a canonical link tag or a resource URL from it) without proper escaping — and that response then gets cached — **every subsequent visitor to the homepage receives the attacker's injected script**, executed in their own browser session, with zero interaction required beyond visiting the normal site. This is XSS delivery *without* needing to trick anyone into clicking a crafted link at all — arguably a more dangerous delivery mechanism than a typical reflected XSS payload.

5. Exploiting Fat GET Requests

Some applications process a request body even on a `GET` request (a "fat GET") — and body parameters are frequently **unkeyed** even when equivalent query-string parameters would be keyed:

```
GET /?param=1 HTTP/1.1
Host: vulnerable-website.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 26
```

```
param=<script>alert(1)</script>
```

If the body parameter influences the response the same way the query parameter would, but isn't part of the cache key, you've found another poisoning vector — this is exactly why your Prevention section (10) explicitly calls out disabling fat GET support as a hardening step.

6. Cache Key Manipulation via Parameter Cloaking / Parser Discrepancies

Different components in the request-handling chain (CDN, reverse proxy, cache, origin server) can **parse the same URL differently** — a parser discrepancy directly analogous to the request smuggling and OAuth `redirect_uri` bypass techniques from earlier guides.

```
Example - parameter cloaking:
/page?utm_content=x&fake=y

If the CACHE only considers utm_content= as keyed, but the ORIGIN
SERVER parses fake=y as a real parameter influencing the response,
an attacker can smuggle a payload through the "fake" parameter
without ever changing the cache key at all
```

This same parser-discrepancy principle extends to the **Host header and request line themselves** — components almost always included in the cache key, but which get parsed, transformed, and normalized along the way, creating gaps where genuinely different requests can be made to **collide** on the same cache key.

7. Cache Probing — Confirming a Poisoning Attempt Actually Worked

```
1. Send your poisoning payload request
2. Wait briefly, then send a CLEAN, ordinary request for the same
   URL (no special headers, no cache-buster) from a DIFFERENT
   session/browser
3. If the clean request's response contains YOUR injected payload,
   the cache is confirmed poisoned - any visitor requesting that
   exact cache key now receives your malicious response
```

8. Multiple/Duplicate Header Handling

Servers, caches, and CDNs sometimes disagree on **which** value to use when a header is duplicated in a single request:

```
X-Forwarded-Host: legitimate-site.com
X-Forwarded-Host: attacker-site.com
```

If the cache reads the **first** value (or ignores the header as ambiguous) while the origin server reads the **last** value, you've found another exploitable discrepancy — same underlying class of bug as Section 6, just via header duplication instead of parameter naming.

9. Cache Poisoning DoS (CPDoS)

Beyond content injection, cache poisoning can be used purely for denial of service — deliberately causing the origin server to generate an **error response** (via an oversized header, an unusual character, a malformed value) and getting that **error page cached** in place of the legitimate content. Every subsequent visitor then receives a broken error page instead of the real site, until the cache entry expires or is manually purged — a genuinely disruptive attack requiring no XSS/injection sophistication at all, just the ability to trigger a server-side error via an unkeyed input.

PART 2: Web Cache Deception

10. The Core Concept — Inverted From Poisoning

Where poisoning injects *malicious* content into the cache for others to receive, deception tricks the cache into storing *legitimate, sensitive, user-specific* content — which the **attacker** then retrieves by requesting the same cache key themselves.

```
1. Victim is logged in, requests a genuinely sensitive, dynamic,
   personal page (their account/profile page)
2. Through a CRAFTED URL, the attacker tricks either the victim's
   browser OR the caching layer into treating this sensitive
   response as if it were a CACHEABLE STATIC RESOURCE
3. The cache stores the victim's private response under a
   predictable/guessable cache key
4. Attacker requests that SAME cache key directly, and receives
   the victim's cached private data - session tokens, API keys,
```

```
    personal information - straight from the cache, no
    authentication required on the attacker's end at all
```

11. Why Cache Rules Create This Gap

Caching layers decide what's cacheable largely based on **URL path patterns**, since dynamic/sensitive content shouldn't be cached (it varies per user and may contain private data), while static resources (CSS, JS, images) are cached aggressively since they're identical for everyone. Two common rule types create the exploitable gap:

```
Static file extension rules  → match based on the extension:
                                   /profile.css, /account.js
Static directory rules         → match based on a URL PATH PREFIX:
                                     everything under /static/, /assets/
```

The vulnerability arises when these pattern-matching rules can be **tricked** into classifying a genuinely dynamic, sensitive URL as if it were one of these static patterns.

12. Classic Exploitation — Path Confusion via Extension Appending

```
Legitimate sensitive URL:  /account/settings
                            (correctly NOT cached - dynamic, per-user)

Attacker-crafted URL:       /account/settings/nonexistent.css
```

If the **caching layer** sees the `.css` extension and decides "this is a static resource, cache it" — but the **origin server** ignores the trailing `/nonexistent.css` portion entirely and serves the *exact same* sensitive `/account/settings` content regardless — you get a parsing mismatch: the cache stores the victim's private page content, keyed under a URL the attacker can predict and request directly themselves.

```
Attacker delivery: trick the victim into visiting
  https://vulnerable-website.com/account/settings/nonexistent.css
  (via a phishing link, or embedded as an <img> tag on a page the
  victim visits — no click needed, just the browser attempting to
  load the "image")
```

```
Attacker retrieval: attacker separately requests the SAME exact URL
  themselves and receives the victim's now-cached private response
```

13. Delimiter-Based Path Confusion — Requiring No User Interaction At All

More advanced variants use special delimiter characters (like `#`) that have meaning to the **cache** but get handled differently — or ignored entirely — by the **origin server**:

```
GET /account/settings#/fake.css
```

If a character like this creates a parsing discrepancy severe enough, the attacker doesn't need the poisoned path to have a recognizable static extension at all, and — significantly — **doesn't need any victim interaction whatsoever**, since (much like poisoning) the exploit can sometimes be delivered in a way that doesn't require tricking a specific victim into clicking anything. This blurs directly into full cache poisoning territory, and demonstrates why the two categories, while conceptually distinct, share the same underlying root cause: **parser discrepancies between the cache and the origin server.**

14. Real-World Impact

```
Token/API key hijacking     → sensitive tokens embedded in what should
                                be a private, dynamic page get cached
                                and become retrievable by anyone who
                                knows/guesses the crafted URL
Full account takeover           → if the leaked data includes session
                                    tokens or credentials
Personal data exposure             → any personally identifiable
                                      information rendered on the
                                      "deceived" cached page
```

15. Testing Methodology for Deception

```
1. Identify sensitive, dynamic, authenticated pages (account
   settings, profile, order history, anything with a token/API
   key visible in the rendered page)
2. Append plausible static-resource patterns to the URL path:
   /sensitive-page/fake.css
   /sensitive-page/fake.js
```

```
   /sensitive-page;fake.css      (path parameter syntax, some servers)
   /sensitive-page%2f..%2ffake.css   (encoded traversal tricks)
3. Check whether the ORIGIN SERVER still returns the FULL sensitive
   content despite the appended fake extension (confirms the origin
   ignores/strips it)
4. Check whether that response GETS CACHED - request the same
   crafted URL again from a DIFFERENT, unauthenticated session; if
   you receive the FIRST session's private content, deception is confirmed
5. Try delimiter-based variants (#, ;, encoded characters) if simple
   extension-appending doesn't trigger caching
```

---

16. Prevention — Web Cache Poisoning

```
- NEVER rewrite the cache key based on unkeyed input processing -
  if a header/parameter genuinely needs to influence behavior,
  either include it properly in the cache key, or better: REWRITE
  THE REQUEST ITSELF (normalize/strip the input) rather than trying
  to selectively key around it - achieves the same performance
  benefit with far less risk of a parsing gap being exploited
- Disable support for fat GET requests (Section 5) if not genuinely
  needed
- Ensure consistent parsing of the Host header and request line
  across every layer in the chain (CDN, proxy, origin) - the
  discrepancies between these layers ARE the vulnerability
- Patch even "unexploitable" reflected issues (self-XSS, encoded-only
  XSS, reflection only inside a resource file) as defense-in-depth -
  what's unexploitable via normal browser interaction can become
  fully exploitable once combined with cache poisoning's
  no-victim-interaction delivery model
- Treat every unkeyed input that influences a response as a
  potential vulnerability, not just the ones you've already found
  a working exploit for
```

17. Prevention — Web Cache Deception

```
- Ensure the ORIGIN SERVER and the CACHING LAYER agree precisely
  on how a given URL path is parsed - eliminate any discrepancy
  between what each layer considers "the resource being requested"
- Configure caching rules to match based on VERIFIED CONTENT TYPE
  in the actual response, not just a pattern-matched URL suffix/prefix
- Never cache responses that include authentication-sensitive
  headers, tokens, or personal data, REGARDLESS of what the request
  URL happens to look like
- Explicitly strip/normalize trailing path segments and delimiter
  characters BEFORE the caching decision is made, so a crafted
  suffix can't smuggle a sensitive page past a static-resource rule
- Set explicit Cache-Control: private / no-store headers on any
  genuinely dynamic, user-specific response, as a direct instruction
```

```
   overriding whatever pattern-based inference the cache might
   otherwise apply
```

18. Practical Lab Example

```
PortSwigger's Web Security Academy has dedicated lab sets for BOTH
topics separately - "Web cache poisoning" and "Web cache deception"
- with the deception labs specifically built around research first
presented at Black Hat USA 2024, and the poisoning labs stemming
from the original 2018/2020 research papers referenced throughout
this guide.

Suggested practice order:
1. Web cache poisoning: "unkeyed header" lab first (Section 4's
   exact X-Forwarded-Host XSS pattern) - install Param Miner and
   practice the discovery workflow from Section 3 before jumping
   to the payload
2. Web cache poisoning: fat GET / parameter cloaking labs (Sections
   5-6) once the basic pattern clicks
3. Web cache deception: static-extension path confusion lab
   (Section 12) - the clearest illustration of the "trick the cache,
   not the server" inversion from poisoning
```

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-444 (HTTP Request/Response Smuggling — the parser-discrepancy family both attacks share), CWE-524 (Information Exposure Through Caching, specifically for deception)
OWASP Top 10:2025	A02:2025 Security Misconfiguration (caching rules/behavior are a configuration surface) — poisoning's XSS payload delivery also touches A05:2025 Injection depending on the resulting payload type
CVSS	Poisoning can reach Critical when it delivers XSS/defacement at scale with zero victim interaction (UI:N is a genuinely accurate metric here, unlike most XSS); deception severity depends entirely on what sensitive data gets exposed, often High when tokens/session data leak
Cyber Kill Chain	Delivery (poisoning turns the site itself into the delivery mechanism, no phishing link needed) → Exploitation (the payload executing in victims' browsers)

Framework	Mapping
MITRE ATT&CK	T1189 (Drive-by Compromise) for poisoning's automatic-delivery XSS; deception overlaps with T1552 (Unsecured Credentials) once tokens are recovered from the cache

Quick Reference Cheat Sheet

```
CACHE POISONING (inject malicious content INTO the cache):
1. Find unkeyed inputs (Param Miner, or manual header testing +
   cache-buster query param)
2. Confirm reflection: X-Forwarded-Host: canary-value → check response
3. Weaponize: inject XSS/redirect payload via the unkeyed input
4. Confirm poisoning: clean request from a separate session/browser
   should now return YOUR payload

CACHE DECEPTION (trick the cache into storing PRIVATE content):
1. Append fake static extensions to sensitive dynamic URLs:
   /account/settings/x.css
2. Confirm origin server ignores the suffix, returns full sensitive content
3. Confirm the response GETS CACHED under that crafted URL
4. Retrieve it yourself, unauthenticated, using the same crafted URL

Root cause shared by both: parser/rule discrepancy between the
CACHE's view of a request and the ORIGIN SERVER's view of the same request
```